LEARNING BLOCKCHAIN · STUDY GUIDE

Day 01 — Hashing & SHA-256

Date: Thursday 15 October 2026 · **Phase:** Foundations (Day 1 of 15) · **Author:** Manish Khadka
Code: github.com/Khadka1996/Learning-Blockchain/tree/main/day-01 · 33 tests passing

Learning objectives

After this day you should be able to:

1. Explain in plain words what a cryptographic hash function is and name its six key properties.
2. Compute SHA-256 of text, bytes and files in Node.js and check results against official NIST test vectors.
3. Measure the **avalanche effect** (≈ 50 % of output bits flip for a one-character change).
4. Explain why hashes are one-way, and how brute-forcing "leading zeros" becomes **Proof of Work**.
5. Spot the input-handling bugs that break hashing code in practice: number parsing, malformed Unicode, file races.

The concept, step by step

Step 1 — A fingerprint for data

A hash function takes **any** input — a word, a payment, a 2 GB file — and returns a short, fixed-size output called a *digest*. SHA-256 always returns 256 bits, written as 64 hexadecimal characters. Like a fingerprint, the digest is tiny compared with the original, belongs to exactly one input in practice, and cannot be used to rebuild the input.



Figure 1 — any input, one fixed-size fingerprint. There is no path back from the digest to the input.

Step 2 — The six properties that blockchains rely on

Property	Meaning	Why a blockchain needs it
Deterministic	Same input → same output, always	Thousands of computers can check the same data independently
Fixed size	Always 256 bits	A block can point to the previous block with a 32-byte reference
Fast	Cheap to compute	Verifying is quick for everyone
Avalanche effect	Tiny change → ~50 % of bits flip	Tampering is instantly visible

Preimage resistant	Cannot work back from digest to input	Enables Proof of Work and commitments
Collision resistant	Infeasible to find two inputs with one digest	The digest can safely stand in for the data

Step 3 — The avalanche effect, measured

Changing `"Manish sends 10 coins"` to `"Manish sends 11 coins"` — one digit — changed **137 of 256 bits (53.5 %)**. Repeating the experiment 10,000 times with random messages gave an average of **127.93 bits (49.97 %)**, with a minimum of 99 and a maximum of 156. Each output bit behaves like a fair coin toss, so theory predicts 128 ± 8.

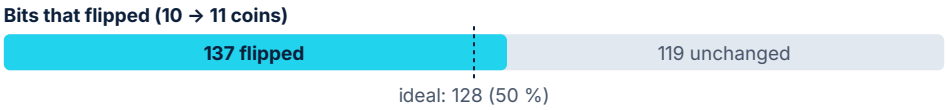


Figure 2 — one edited character changes more than half the fingerprint.

Step 4 — One-way, so the only strategy is guessing

To find an input whose hash starts with *n* zero hex digits, there is no formula: you try input after input. Each hex digit has 16 possible values, so each extra zero makes the search about **16× harder** (expected 16^n attempts). Bitcoin mining is this same search at enormous scale, and the "difficulty" is simply how many zeros (more precisely, how small a number) the hash must be.

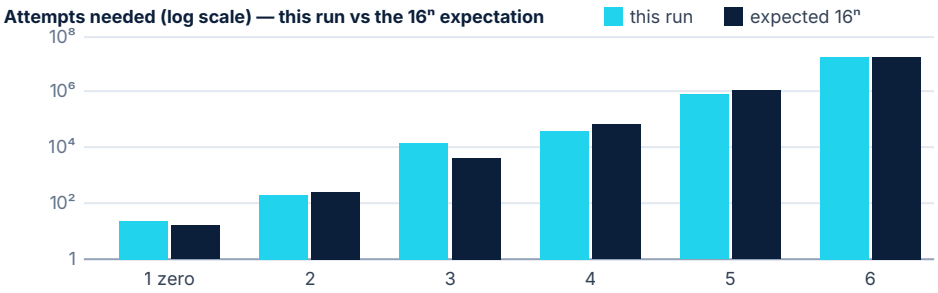


Figure 3 — each extra zero multiplies the work by ~16. Single runs vary (luck), the trend never does.

Step 5 — Why this builds a chain (preview of Day 02)

Each block stores the hash of the block before it. Edit any old block and its hash changes, which breaks the pointer in the next block, which breaks the next one, and so on. Combine that with Proof of Work and rewriting history means redoing all the guessing for every later block.



Figure 4 — every block carries the previous block's fingerprint (illustrative values).

Key code walkthrough

All logic lives in `src/lib.js` as small, pure, validated functions; the CLI scripts only parse arguments and print. The snippets below are the real code after the evening audit.

1. Hashing — with input checks that prevent silent collisions

```
// Only well-formed strings, Buffers and Uint8Arrays are accepted.
function assertHashable(input) {
  if (typeof input !== "string" && !(input instanceof Uint8Array)) {
    throw new TypeError(`sha256 expects a string, Buffer or Uint8Array, …`);
  }
  // A lone UTF-16 surrogate has no UTF-8 form; Node would silently turn it into U+FFFD,
  // so "\uD800", "\uDC00" and "\uFFFD" would all share one hash (audit finding E1).
  if (typeof input === "string" && !isWellFormed(input)) {
    throw new TypeError("sha256 expects a well-formed Unicode string …");
  }
}
function sha256Bytes(input) {
  assertHashable(input);
  return createHash("sha256").update(input).digest();   // 32-byte Buffer
}
```

2. Measuring the avalanche — counting differing bits

```
function bitsDifferent(a, b) {             // Hamming distance of two 32-byte digests
  let diff = 0;
  for (let i = 0; i < a.length; i++) {
    let x = a[i] ^ b[i];                  // XOR: 1 wherever the bits differ
    while (x) { diff += x & 1; x >>>= 1; }  // count the 1s in this byte
  }
  return diff;                             // 0 … 256
}
```

3. Brute force — the idea behind Proof of Work

```
function findLeadingZeros(zeros, { prefix = "manish-", maxAttempts = 2 ** 32 } = {}) {
  const z = parseZeros(zeros);             // whole number 1–8 only (F1–F3, E4)
  if (typeof prefix !== "string") throw new TypeError("prefix must be a string");  // E5
  const target = "0".repeat(z);
  for (let nonce = 0; nonce < maxAttempts; nonce++) {   // always terminates (F3)
    const hash = sha256Hex(`${prefix}${nonce}`);
    if (hash.startsWith(target)) return { found: true, nonce, hash, attempts: nonce + 1 };
  }
  return { found: false, attempts: maxAttempts };
}
```

4. File fingerprints — check the file you actually read

```
// Open ONCE, then check the open handle. Checking the path and re-opening it left a
// gap where the file could be swapped for a pipe that never ends (audit finding E2).
const handle = await fs.promises.open(path, O_RDONLY | O_NONBLOCK);
const info = await handle.stat();
if (!info.isFile()) { await handle.close(); throw new Error(`not a regular file: ${path}`); }
handle.createReadStream()
  .on("data", (chunk) => { bytes += chunk.length; hash.update(chunk); })  // size = bytes read
  .on("end",  () => resolve({ bytes, hex: hash.digest("hex") }));
```

Results from the run

Experiment	Result
"10 coins" → "11 coins"	137 / 256 bits (53.5 %)
10,000 random 1-char edits	avg 127.93 (49.97 %)
Fewest / most bits flipped	99 / 156
Bitcoin genesis id (exercise)	10 leading zeros
Tests · lint · audit	33 pass · clean · 0 vulns

Zeros	Attempts (run)	Expected 16^n
1	23	16
2	193	256
3	13,699	4,096
4	37,213	65,536
5	742,576	1,048,576
6	16,325,200	16,777,216

The difficulty table is now generated by `npm run preimage:table`, and a test pins the first rows, so the notes and the code cannot drift apart. Five zeros took 742,576 guesses (about 2 seconds on a laptop); six took 16.3 million (≈ 44 s).

QA & security findings

The morning review found and fixed 8 issues (F1–F9; F7 accepted as documented behaviour). The evening audit — run as an outside reviewer — found 5 more. Every one got a failing test first, then the fix.

ID	Severity	Problem	Fix
F1	**Medium**	`preimage.js abc` → `NaN` zeros → every hash "matched" — a fake success.	Strict validation, exit code 1
F3	**Medium**	65 zeros (more than a hash has) → infinite loop.	Cap at 8 + `maxAttempts`
F8	**Medium**	Fingerprinting `/dev/zero` streamed forever (denial of service).	Regular files only
E1	**Medium**	**Real collision:** `"\uD800"`, `"\uDC00"` and `"\uFFFD"` all hashed to `83d544cc…` because Node replaces lone surrogates before hashing.	Reject malformed strings; 5,000-case fuzz test
E2	**Low**	Time-of-check / time-of-use race: path checked, then re-opened.	Open once, check the handle
E3	**Low**	`hash.js ""` hashed the demo text instead of the empty string.	Default only when no argument
E4	**Low**	`--stats 0x10` ran 16 trials; `parseZeros([5])` was accepted.	One strict `toWholeNumber()`
E5	**Info**	Non-string `prefix` became `"[object Object]"`.	`TypeError`

Known issues (accepted): F7 — visually identical Unicode text can hash differently, so normalise user text with `normalize("NFC")`; E6 — invalid UTF-8 in command-line arguments is replaced by

Node before our code runs; E7 — 8 zeros is allowed and takes hours by design. Full details: `day-01/QA.md`.

Common mistakes

- **Hashing the hex text instead of the bytes.** `sha256("ab")` and `sha256(Buffer.from("ab","hex"))` are different inputs. Bitcoin hashes raw bytes.
- **Forgetting byte order.** Bitcoin shows block and transaction ids byte-reversed; compare like with like.
- **Assuming strings are bytes.** Text must be encoded (UTF-8) first; malformed or un-normalised text gives surprising hashes (E1, F7).
- **Trusting `Number()` on user input.** It happily accepts `"0x10"`, `"1e3"` and `" "` (→ 0).
- **Hashing without a salt** when the input space is small (votes, moves, PINs): an attacker just hashes every option.
- **Using a plain hash as a password store or a MAC.** Use a slow password hash (scrypt, Argon2) or HMAC instead.

Five interview questions

1. What is the difference between preimage resistance and collision resistance?

Preimage resistance: given a digest, you cannot find *any* input that produces it (≈ 2^{256} work for SHA-256). Collision resistance: you cannot find *any two* inputs with the same digest (≈ 2^{128} work, because of the birthday paradox). Collisions are easier because the attacker chooses both inputs.

2. Why does Bitcoin hash block headers twice (double SHA-256)?

Satoshi never documented the reason; the common explanation is defence-in-depth against length-extension and other weaknesses of the Merkle–Damgård construction. In practice it gives the `hash256` function used for block and transaction ids.

3. How does Proof of Work use a hash function?

Miners change a nonce in the block header until `hash256(header)` is below a target. Because the hash is one-way and unpredictable, the only method is guessing; the expected number of guesses is set by the difficulty, and anyone can verify the result with one hash.

4. A teammate hashes user names to create unique ids. What can go wrong?

Visually identical names can have different bytes (Unicode normalisation, F7), and malformed strings can collapse into the same bytes (E1). Normalise to NFC, reject malformed text, and hash a canonical byte encoding.

5. Why is `sha256(secret ‖ message)` not a safe way to authenticate a message?

SHA-256 is vulnerable to length extension: knowing the digest and the length, an attacker can compute the hash of `secret ‖ message ‖ padding ‖ extra` without the secret. Use HMAC-SHA-256 (or SHA-3, which is not affected).

Exercises (in `day-01/exercises/`)

Level	Exercise	Goal
Easy	Spot the tampered record	Re-hash each record and return the ids whose text no longer matches its published hash. `npm run ex:easy`
Medium	Verify Bitcoin's genesis block	Implement double SHA-256 and reproduce block 0's id `000000000019d668…` (10 leading zeros) from its 80-byte header. `npm run ex:medium`
Challenge	Commit–reveal	Build salted commitments for rock–paper–scissors, show the 3-guess attack on unsalted ones, verify with a constant-time compare. `npm run ex:challenge`

Cheat sheet — Day 01

SHA-256 output	256 bits = 32 bytes = 64 hex characters
Node.js	`crypto.createHash("sha256").update(data).digest("hex")`
Empty-string hash	`e3b0c442…7852b855` (a quick sanity check)
Avalanche	1 changed bit/char → ~128 of 256 bits flip
Brute force n hex zeros	$\approx 16^n$ attempts on average
Security levels	preimage $\approx 2^{256}$ · collision $\approx 2^{128}$
Bitcoin ids	`hash256 = SHA-256(SHA-256(x))`, shown byte-reversed
Text input	encode UTF-8 · reject lone surrogates · `normalize("NFC")`
Secrets	HMAC for messages · salt for small choices · scrypt/Argon2 for passwords

References

1. NIST FIPS 180-4, *Secure Hash Standard (SHS)* — https://csrc.nist.gov/pubs/fips/180-4/upd1/final
2. NIST, *Cryptographic Standards and Guidelines — Example Values (SHA-256 test vectors)* — https://csrc.nist.gov/projects/cryptographic-standards-and-guidelines/example-values
3. S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (2008) — https://bitcoin.org/bitcoin.pdf
4. A. Antonopoulos & D. Harding, *Mastering Bitcoin*, 3rd ed. (open edition) — https://github.com/bitcoinbook/bitcoinbook
5. P. Rogaway & T. Shrimpton, *Cryptographic Hash-Function Basics*, FSE 2004 — https://web.cs.ucdavis.edu/~rogaway/papers/relates.pdf
6. Bitcoin Developer Guide — Block Chain — https://developer.bitcoin.org/devguide/block_chain.html
7. ethereum.org — Proof-of-work (PoW) — https://ethereum.org/en/developers/docs/consensus-mechanisms/pow/
8. MDN — `String.prototype.isWellFormed()` (lone surrogates) — https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/isWellFormed
9. MITRE CWE-367, *Time-of-check Time-of-use (TOCTOU) Race Condition* — https://cwe.mitre.org/data/definitions/367.html